Safe Edges

2025

Smart Contract Audit REPORT

ASSESSED ON Nov, 2025

Clique

Security Assessment

SAFEEDGES.IN

Clique EIP7702 Daft Audit Report

Table of Contents

Executive Summary

Item	Details
Scope	`https://github.com/CliqueOffi-cial/clique-lock/blob/main/src/hook/Lin-earPenaltyHook.sol` , `https://github.com/CliqueOffi-`

	`cial/clique-lock/blob/main/src/hook/LinearPenaltyHook.sol`
Timeframe	e.g., Nov, 2025
Methodology	Manual code review, static analysis, and best practice verification
Severity Summary	HIGH
Overall Risk	LOW
Status	Under Review / Pending Fixes

>

Techniques and Methods

Code quality checks

Best practices & documentation review

Gas efficiency analysis

Reentrancy & ERC compliance checks

Analysis Types

Structural Analysis – Reviewed design patterns & contract structure

Static Analysis – Automated scanning for vulnerabilities

Manual Analysis – Line-by-line code review

Gas Consumption – Optimization analysis

Tools: Remix IDE, Foundry, Slither, Mythril, Solhint

Severity Levels

Critical

Issues that can completely compromise protocol integrity or allow direct theft/loss of funds. Must be fixed before deployment.

High

Exploitable vulnerabilities affecting critical functionality, potentially causing loss of assets, broken pools, or bypassed core logic. Must be addressed immediately.

Medium

Weaknesses causing errors, inconsistent behavior, or revenue loss under certain conditions. Should be fixed but not catastrophic.

Low

Minor issues with limited impact, such as edge-case bugs or inefficient gas usage. Worth fixing for reliability.

Informational

Cosmetic or documentation issues. Do not pose direct threats but may confuse users or developers.

Issue Status

Open	Resolved
Security vulnerabilities identified that must be resolved and are currently unresolved.	Security vulnerabilities that have been identified and successfully fixed or mitigated.
Acknowledged	**Partially Resolved**

Vulnerabilities which have been acknowledged but are yet to be resolved.	Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

ID	Title	Severity
L-01	Weak distributor access assumption in `LinearPenaltyHook.execute`	LOW
L-02	Weak replay protection in `DelegateMultiExec.multiExec`	LOW
L-03	Use rounding up in `LinearPenaltyHook.getPenalty` to better discourage early claimers	LOW
I-01	Potential griefing risk in `DelegateMultiExec.multiExec`	INFORMATIONAL
G-01	Avoid zero assignment in `for` loop counters for efficiency	GAS

Detailed Findings

[L-01] Weak distributor access assumption in `LinearPenaltyHook.execute`

Description The `execute` function in `LinearPenaltyHook` computes and returns a penalty based on the provided allocation amount and timestamp.

However, the logic implicitly assumes that only the authorized `distributor` contract will invoke this function, as it derives the `_configurationId` via `_root`. In practice, there are no access control checks enforcing that only the distributor can call execute.

```
function execute(
 address,
 address _recipient,
 bytes32 _root,
 uint256 _allocatedamount,
 bytes memory _extra
) external returns (uint256 _consumed) {
 ClaimExtra memory claimExtra = abi.decode(_extra, (ClaimExtra));
@>>        bytes32 _configurationId = _distributor().configurationId(_root);
```

Anyone can call this function to bypass the implicit `_distributor()` caller check and preview the penalty. Since, the function does not perform any state changes, this does not directly damage the protocol.

Impact While this does not lead to any exploit or unauthorized state modification, it represents a weak trust assumption regarding the source of calls and could expose internal logic to unintended external queries.

Recommended mitigation If this function is intended to be used exclusively by the distributor, explicitly restrict its access.

[L-02] Weak replay protection in `DelegateMultiExec.multiExec`

Description `multiExec` constructs an ECDSA-signed digest that includes a `nonce` field, but the contract does not persist or check any on-chain state for that nonce. Because of that, a previously observed valid signed message can be re-submitted repeatedly and will pass verification each time, allowing the same batch of calls to be executed multiple times.

It is possible that the nonce is being managed off-chain; however, if that is not the case, it should be incremented or updated on-chain after each successful execution to prevent reuse.

Impact A signed batch can be replayed using previously valid call data if the nonce is not properly updated or tracked.

Recommended mitigation implement an on-chain sequential nonce that increments after each successful execution to ensure every signed batch can be used only once.

[L-03] Use rounding up in `LinearPenaltyHook.getPenalty` to better discourage early claimers

Description The `getPenalty` function calculates the claim penalty proportionally based on the remaining vesting duration:

```
uint256 penalty = (_allocatedamount * restTime) / totalTime;
```

By default, Solidity's integer division rounds down, meaning the computed penalty may be slightly lower than the mathematically precise ratio.

Since the purpose of the penalty mechanism is to discourage early claims, this rounding direction slightly favors early claimers.

Impact Minor underestimation of penalties for early claimers due to rounding down.

Recommended mitigation Use rounding up to ensure the penalty fully reflects the intended ratio, for example :

```
penalty = FixedPointMathLib.mulDivUp(_allocatedamount, restTime, totalTime);
```

[I-01] Potential Gas griefing risk in `DelegateMultiExec.multiExec`

Description `multiExec` iterates over an array of user-provided calls and executes each via:

```
(bool success, bytes memory returnData) = callEntry.target.call{ value: callEntry.value
}(callEntry.data);
```

Because call is invoked without an explicit gas: stipend, it forwards all remaining gas to the `target`. A malicious or poorly implemented target (for example, a contract fallback or receive() that runs an expensive loop or may return large returnData) can consume most or all gas for the transaction. If `callEntry.allowFailure`

is false, such consumption can cause the whole multiExec to revert; if allowFailure is true, it still wastes the relayers gas and can cause subsequent calls to fail or be starved of gas.

Impact This issue can cause a denial-of-service condition, allowing a single malicious target to halt or disrupt the execution of the entire batch.

Recommended mitigation Avoid storing or returning large returnData values in memory to minimize gas overhead from memory copying. Additionally, if feasible, restrict multiExec execution to trusted or whitelisted target contracts whenever possible.

[G-01] Avoid zero assignment in `for` loops counters for gas efficiency

Description: To optimize gas efficiency in a for loop, it is recommended to avoid explicitly setting the counter to zero, as it is already initialized to zero by default. Removing unnecessary variable assignments will further enhance the contract's gas efficiency.

```
-   for (uint256 i = 0; i < ...; ) {...}
+   for (uint256 i; i < ...; ) {...}
```

Safe Edges

REVOLUTIONIZING BLOCKCHAIN AUDITS

Founded in 2023, Safe Edges is the largest blockchain security company that ensures the safety and reliability of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative technology, we support the success of our clients with best-in-class security, all whilst realizing our overarching vision: ensuring blockchain remains secure and trustworthy.

THANK YOU FOR WORKING WITH US

SAFEEDGES.IN